
4-Actuator Exo-Arm System

Trajectory Control: from Simple Hardware PD
to Model-Based Computed Torque

AK80-kv60 (shoulder) · AK60-kv80 (elbow) · 2× Damiao (exo) · AS5048A encoder

Repo `exo-actuator-sensor-drivers`

May 2026

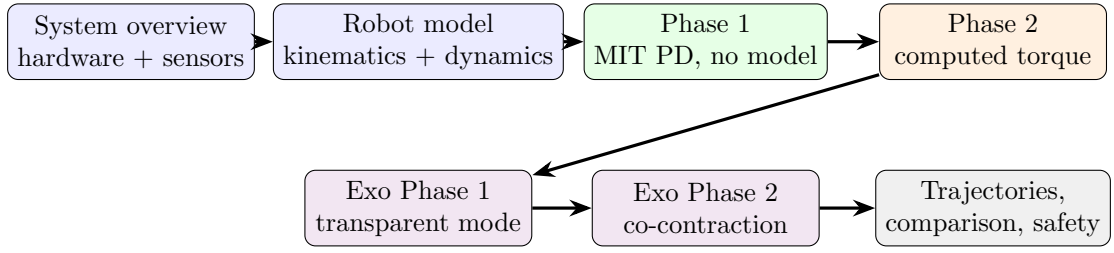
✓ Key Result

- **Phase 1 (start here):** MIT on-board PD requires *no mass/CoM knowledge*. Shoulder and elbow track joint-space references; exo Damiao motors hold transparent (zero-torque) mode. AS5048A provides accurate absolute elbow angle independent of CAN round-trip latency.
- **Phase 2 (after identification):** Computed-Torque inversion of the full nonlinear dynamics cancels inertia and Coriolis coupling (gravity is negligible: arm moves in the horizontal XY plane, gravity acts in $-Z$). Series Elastic Actuator (SEA) compensation at the elbow accounts for cable-spring lag between motor drum position and joint angle.

Contents

1	System Overview	3
1.1	Hardware inventory	3
1.2	Geometric and physical parameters	3
1.3	Mechanical structure and cable routing	4
1.4	Multi-bus topology	4
2	Robot Model (Reference for Phase 2)	5
2.1	Forward kinematics	5
2.2	Jacobian	5
2.3	Equations of motion	5
2.4	Inverse kinematics	6
3	Cable Transmission Model	6
3.1	Drive cable (SEA on elbow, joint 2)	6
3.2	Exo cable (single Damiao motor — centred elbow pulley)	7
4	First-Run Commissioning: How to Bring Up the Manipulator	8
4.1	How a Traditional Robot Is Made to Work	8
4.2	Our Arm: Two Motors, Two Joints, One Encoder	8
4.3	Case A: Drive Cable (No Spring) — Manual Home Positioning	9

4.4	Case B: Exo Cable (Spring via Key Rings) — Damiao Motor	14
4.5	Why Cable Compliance Changes Everything vs. Traditional Control	16
5	Phase 1 — Manipulator Only (MIT PD, No Model)	16
5.1	MIT on-board PD	17
5.1.1	Shoulder vs. Elbow: What MIT Sees	17
5.1.2	Case 1: Direct Cable (No Spring) — Thought Experiment	17
5.1.3	Case 2: Elastic Cable (With Spring) — Actual System	18
5.2	AS5048A encoder as elbow angle source	19
5.3	Complete Phase 1 control loop (Arm Only)	20
5.4	Phase 1 commissioning checklist	21
6	Phase 2 — Manipulator Only (Computed Torque)	21
6.1	Parameter identification workflow	22
6.2	Why the CT law has this form	22
6.3	Computed Torque Control law	24
6.4	SEA cable compensation at the elbow	25
6.5	Hybrid CT + inner MIT damping	25
6.6	Obtaining the real model from the robot in practice	26
6.7	Using Onshape and PyDrake to obtain the dynamic model	27
7	Phase 1 with Exo — Transparent Mode	28
7.1	Exo Damiao motors — transparent mode	28
7.2	Complete 4-actuator Phase 1 loop (arm + transparent exo)	29
7.3	Exo transparent commissioning checklist	30
8	Phase 2 with Exo — CT + Co-contraction	30
8.1	Exo co-contraction activation	30
9	Trajectory Generation	31
9.1	Minimum-jerk point-to-point	31
9.2	Circle trajectory	31
10	Controller Comparison	32
11	Safety Constraints	32
12	Simulation Validation Bridge	32



Lecture story: first understand the hardware and model, then show why Phase 1 works without identification, then motivate Phase 2 as the model-based upgrade, and only after that introduce the exo modes.

Figure 1: Lecture roadmap for the printed notes. This gives the audience a high-level map of the narrative before the technical details begin.

1 System Overview

The bench robot is a **2-DOF planar arm** with two joints:

- **Shoulder** (q_1) — **direct drive**. AK80-kv60 motor is *housed inside the 3D-printed base housing*. Its output shaft connects directly to Link 1; there is no cable between the motor and the link.
- **Elbow** (q_2) — **cable SEA**. AK60-kv80 motor is *mounted on Link 1* (near the elbow end, inside the link structure) and drives the joint through two antagonistic cables. The cables route from the motor drum through 623zz idler bearings along Link 1 and wrap around the **elbow joint drum** ($r_p=47.75$ mm) fixed on the elbow shaft. A spring element k_s on the cable forms a Series Elastic Actuator (SEA).

One **Damiao** motor (*second not yet mounted*) adds an **exo cable** to the elbow joint drum, providing additional stiffness when activated. An **AS5048A** absolute magnetic encoder on the elbow joint axis reads the true joint angle, bypassing the cable-spring deflection that corrupts the motor’s CAN feedback.

1.1 Hardware inventory

Joint	Motor	Firmware	Driver file	$T_{\max}$	Notes
Shoulder (q_1)	CubeMars AK80-kv60	V1.x	ak_v1.bus.py	32 N·m	Direct drive ; inside base housing; s
Elbow (q_2)	CubeMars AK60-kv80	V3.0	ak_v3.bus.py	15 N·m	Cable SEA ; mounted on Link 1; ext
Exo (θ_e)	Damiao DM4310	MIT	damiao.bus.py	10 N·m	1 motor mounted; <code>can_id=0x01</code> ; 2nd
Elbow encoder	AS5048A (14-bit SPI)	—	as5048a.py	—	/dev/spidev0.0 (Jetson Orin)
Elbow encoder	AS5048A (14-bit SPI)	—	as5048a.py	—	/dev/spidev0.0 (Jetson Orin)

△ Important

Two CAN firmwares on one bus. AK80-kv60 (shoulder) uses V1.x firmware with **standard 11-bit CAN frames**; AK60-kv80 (elbow) uses V3.0 firmware with **29-bit extended frames**. Both can coexist on `can1` because CAN hardware distinguishes frame types, but they require *separate* bus wrapper objects (CubeMarsAkV1Bench vs. CubeMarsAkV3Bench).

1.2 Geometric and physical parameters

Symbol	Meaning	Simulation value (Onshape URDF)	Hardware (TBD)
L_1	Upper-arm link length	335 mm	measure
L_2	Forearm link length	380 mm	measure
m_1	Upper-arm mass (incl. motor)	5.313 kg	weigh
m_2	Forearm link length	0.764 kg	weigh
l_{c1}	Upper-arm CoM distance	$L_1/2$ (rod approx.)	balance
l_{c2}	Forearm CoM distance	$L_2/2$ (rod approx.)	balance
r_p	Drum radius (motor output shaft & elbow joint shaft)	47.75 mm	measure
r_{exo}	Exo elbow drum radius (same drum as drive cable)	47.75 mm	measure

1.3 Mechanical structure and cable routing

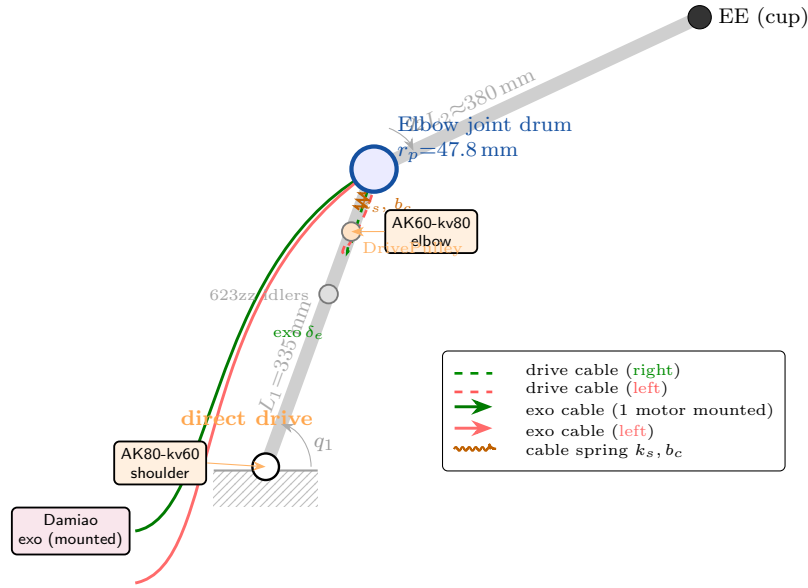
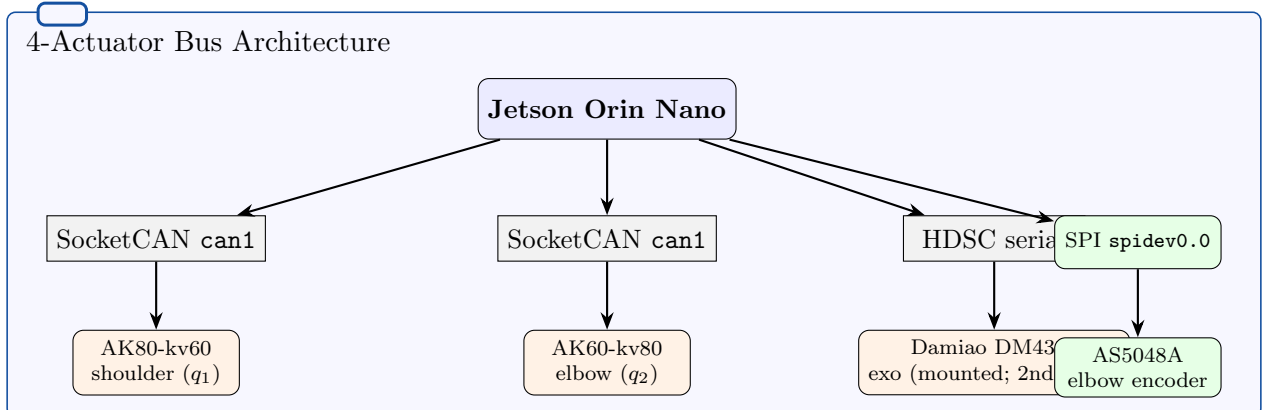


Figure 2: Mechanical structure of the 4-actuator arm (canonical pose: $q_1=70^\circ$, $q_2=-45^\circ$). **Shoulder joint** q_1 : direct-drive AK80-kv60 at the base. **Elbow joint** q_2 : cable-driven via two antagonistic cables. The AK60-kv80 elbow motor (mounted on Link 1, near the elbow end at $\sim 79\%$ of L_1 , inside the link structure) drives a motor drum; cables pass over 623zz idler bearings and wrap around the elbow joint drum ($r_p=47.8$ mm) on the elbow shaft. A zigzag symbol marks the cable spring k_s, b_c (SEA). One Damiao exo motor (second not yet mounted) delivers an exo cable to the elbow joint drum. Both CubeMars motors are enclosed inside the 3D-printed arm structure and are not externally visible.

1.4 Multi-bus topology



Listing 1: Bus instantiation using declarative configs (src/motor_config.py).

```

1 from motor_config import (
2     CubeMarsAkV1BusConfig, CubeMarsAkV1MotorConfig, # shoulder
3     CubeMarsAkV3BusConfig, CubeMarsMotorConfig, # elbow
4     DamiaoBusConfig, DamiaoMotorConfig, # exo
5 )
6 from encoder_config import AS5048AConfig
7
8 # Shoulder: AK80-kv60 (V1.x, standard CAN frames)
9 shoulder_cfg = CubeMarsAkV1BusConfig(
10     channel="can1",
11     motors={"shoulder": CubeMarsAkV1MotorConfig(can_id=0x01, model="AK80-8")},
12 )
13
14 # Elbow: AK60-kv80 (V3.0, extended CAN frames)
15 elbow_cfg = CubeMarsAkV3BusConfig(
16     channel="can1",
17     motors={"elbow": CubeMarsMotorConfig(can_id=0x02, model="AK60-6")},
18 )
19
20 # Exo: 1x Damiao DM4310 mounted (2nd not yet installed)
21 exo_cfg = DamiaoBusConfig(
22     port="/dev/ttyACM0",
23     motors={
24         "exo": DamiaoMotorConfig(can_id=0x01, master_id=0x11),
25     },
26 )
27
28 # Elbow absolute encoder: AS5048A (SPI bus 0, CS 0)
29 enc_cfg = AS5048AConfig(bus=0, device=0, max_hz=1_000_000)

```

2 Robot Model (Reference for Phase 2)

Phase 1 does not use any of this section. Skip ahead to Section 5 for commissioning. Phase 2 requires the physical parameters in the table above.

2.1 Forward kinematics

Let $s_1 = \sin q_1$, $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$, $c_{12} = \cos(q_1 + q_2)$:

$$p(q) = \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} L_1 c_1 + L_2 c_{12} \\ L_1 s_1 + L_2 s_{12} \end{bmatrix}. \quad (1)$$

2.2 Jacobian

$\dot{p} = J(q) \dot{q}$, where

$$J(q) = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix}. \quad (2)$$

Damped pseudoinverse $J^\dagger = J^T(JJ^T + \lambda^2 \mathbf{I})^{-1}$ is used near singularities ($\lambda \approx 0.01$).

2.3 Equations of motion

$$M(q) \ddot{q} + C(q, \dot{q}) \dot{q} + g(q) = \tau. \quad (3)$$

Inertia matrix (uniform-rod model, $c_2 = \cos q_2$):

$$\begin{aligned} M_{11} &= \frac{4}{3}m_1L_1^2 + m_2(L_1^2 + \frac{4}{3}L_2^2 + 2L_1L_2c_2), \\ M_{12} &= m_2(\frac{4}{3}L_2^2 + L_1L_2c_2), \quad M_{22} = \frac{4}{3}m_2L_2^2. \end{aligned} \quad (4)$$

Coriolis/centrifugal ($h = -m_2L_1L_2s_2$):

$$\mathbf{C} = \begin{bmatrix} h\dot{q}_2 & h(\dot{q}_1 + \dot{q}_2) \\ -h\dot{q}_1 & 0 \end{bmatrix}. \quad (5)$$

Gravity. The arm moves in the horizontal XY plane; gravity acts in $-Z$ (perpendicular to the plane of motion). Because no link rotation occurs about a horizontal axis, gravity produces **no torque on either joint**:

$$\mathbf{g}(\mathbf{q}) = \mathbf{0}. \quad (6)$$

This simplifies the equations of motion and the Computed-Torque law: the $\mathbf{g}(\mathbf{q})$ term vanishes, leaving only inertia and Coriolis effects to cancel.

2.4 Inverse kinematics

$$q_2 = \text{atan2}\left(\pm\sqrt{1-D^2}, D\right), \quad D = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}, \quad (7)$$

$$q_1 = \text{atan2}(y, x) - \text{atan2}(L_2s_2, L_1 + L_2c_2). \quad (8)$$

3 Cable Transmission Model

This section describes the cable mechanics for the elbow SEA drive and exo cables. **Joint 1 (shoulder) torque passes through unchanged**; all cable dynamics apply to joint 2 (elbow) only.

3.1 Drive cable (SEA on elbow, joint 2)

The AK60-kv80 motor drives **two antagonistic cables**. Each cable wraps on a **motor drum** fixed to the motor output shaft (radius $r_p = 47.75$ mm). The other end wraps on the **elbow joint drum** fixed to the elbow shaft (same radius r_p), so one full revolution of the motor output shaft reels in $2\pi r_p$ of cable. Both cables are wound in opposite directions, making the drive *antagonistic*: winding one cable slackens the other.

△ Important

Only one cable is ever taut at a time. Both cables are wound on the motor drum in opposite directions. When the motor output shaft rotates in the $+$ direction it *winds* the green cable (shortening it $\rightarrow$ tension) and simultaneously *unwinds* the red cable (lengthening it $\rightarrow$ slack). The reverse is true for the $-$ direction. The two cables therefore *cannot* both be taut simultaneously — whichever side is slack carries zero force. The spring k_s is in series with each cable, so “spring extension” δ is purely a signed scalar: $\delta > 0$ means green cable is loaded; $\delta < 0$ means red cable is loaded.

Choosing motor-side cable displacement as state, with drum radius $r_p = 47.75$ mm (both drums — motor output shaft and elbow joint shaft):

Cable extension (spring deflection):

$$\delta = r_p \theta_m - r_p q_2 = r_p (\theta_m - q_2). \quad (9)$$

$\delta > 0$: green cable is shortened (taut); red cable is slack.

$\delta < 0$: red cable is shortened (taut); green cable is slack.

In **position mode** the state is the cable displacement $l_m = r_p \theta_m$, giving $\delta = l_m - r_p q_2$.

Cable force (unilateral spring-damper): Because cables can only pull, the raw spring force is split into the active cable's tension and the slack cable contributes nothing:

$$F_{\text{raw}} = k_s \delta + b_c \dot{\delta}. \quad (10)$$

$$T_{\text{green}} = \max(F_{\text{raw}}, 0), \quad T_{\text{red}} = \max(-F_{\text{raw}}, 0). \quad (11)$$

At any instant $T_{\text{green}} \cdot T_{\text{red}} = 0$: *exactly one* of the two is non-zero (or both are zero at $\delta = 0$, i.e. no load). The net cable force on the elbow joint drum is $F_{\text{cable}} = T_{\text{green}} - T_{\text{red}}$.

Joint torque:

$$\tau_2 = r_p F_{\text{cable}} = r_p (T_{\text{green}} - T_{\text{red}}) \quad (12)$$

△ Important

The AS5048A reads the **joint** angle q_2^{enc} directly from the elbow shaft. The CAN feedback from the motor reports θ_m — the motor *output shaft* angle (CubeMars QDD firmware exposes the output shaft directly; no division by gear ratio is needed in software). Their difference at rest is zero; under load it equals the spring extension: $\theta_m - q_2^{\text{enc}} = \delta / r_p$. Always use the AS5048A as ground-truth for q_2 in the controller; use the CAN velocity for $\dot{q}_2$ where the encoder does not provide velocity directly.

3.2 Exo cable (single Damiao motor — centred elbow pulley)**△ Important**

Current hardware state: only *one* Damiao motor is mounted. The second motor is not yet installed. The equations below cover both the single-motor (now) and the future two-motor antagonistic case.

The mounted Damiao motor drives one exo cable that wraps around the elbow joint drum (radius $r_{\text{exo}} = r_p = 47.75 \text{ mm}$). The Damiao CAN feedback reports θ_{me} directly (output shaft angle). Cable extension and torque for the mounted motor:

$$\delta_e = r_{\text{exo}} (\theta_{me} - q_2), \quad \tau_{\text{exo}} = +r_{\text{exo}} F_e \quad (\text{pulls} + q_2). \quad (13)$$

Motor commands (single motor):

$$\text{Transparent: } \theta_{me} = q_2 \quad \Rightarrow \quad \delta_e = 0, \tau_{\text{exo}} = 0. \quad (14)$$

$$\text{Activated}(\Delta\theta): \theta_{me} = q_2 + \Delta\theta \quad \Rightarrow \quad \delta_e = r_{\text{exo}} \Delta\theta, \tau_{\text{exo}} = r_{\text{exo}} k_{\text{exo}} \delta_e. \quad (15)$$

Exo torque and stiffness (single motor):

$$\tau_{\text{exo}} = r_{\text{exo}} F_e, \quad k_{\text{eff}} = k_{\text{exo}} r_{\text{exo}}^2. \quad (16)$$

With the second motor installed, stiffness doubles: $k_{\text{eff}} = 2 k_{\text{exo}} r_{\text{exo}}^2$.

✓ Key Result

With $k_{\text{exo}}=500 \text{ N/m}$ and $r_{\text{exo}}=47.75 \text{ mm}$:
 Single motor: $k_{\text{eff}} = 500 \times (0.04775)^2 \approx 1.14 \text{ Nm/rad}$.
 Two motors (future): $k_{\text{eff}} \approx 2.28 \text{ Nm/rad}$.

4 First-Run Commissioning: How to Bring Up the Manipulator

This section describes the exact sequence you must follow to bring the two-motor arm to life for the first time. The exo Damiao motor is left **transparent** (zero-torque mode) throughout Phase 1; we only deal with the shoulder (AK80-kv60) and elbow (AK60-kv80).

4.1 How a Traditional Robot Is Made to Work

A **traditional servo-driven robot** (e.g. Franka, UR5) uses motors with *separate external gearboxes* (harmonic drives, planetary stages) between the motor shaft and the joint axis. There are two encoder positions: one on the motor shaft and one on the joint output shaft.

This is **different from our CubeMars motors**. The CubeMars AK-series are **quasi-direct-drive (QDD)** motors: a small planetary stage is *built inside the motor housing*. The CAN interface reports and accepts the *output shaft* position directly — from the software side there is no visible gear ratio. You send q_d in joint-space and read back q in joint-space. The analogy below is for context only — our shoulder behaves the same way (direct command $\rightarrow$ joint responds), just without an external gearbox.

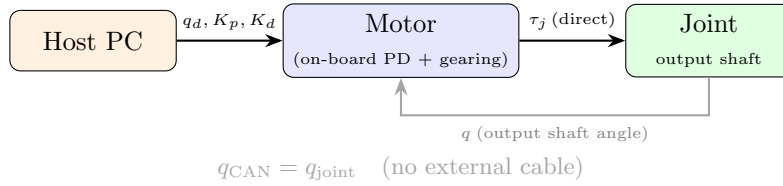


Figure 3: Traditional servo robot (and our **shoulder**): motor sends torque directly to the joint. CAN feedback angle equals joint angle. No spring, no cable lag.

✓ Key Result

In a rigid-transmission robot, $q_{\text{motor}} = q_{\text{joint}}$ always. The PD loop closes on the actual joint position. No spring, no lag, no compliance. High K_p is safe.

4.2 Our Arm: Two Motors, Two Joints, One Encoder

Our arm differs from the traditional case in two ways:

1. **Shoulder** (q_1) behaves like a traditional direct-drive joint. Motor shaft connects rigidly to Link 1; $q_1 = \theta_{m1}$ (CAN reports output shaft angle directly).
2. **Elbow** (q_2) is driven through a **direct steel cable** (no spring in the drive cable path, current setup). Once the cable is wound taut, motor angle $\approx$ joint angle. The AS5048A absolute encoder reads the true joint angle directly from the joint axis and is the primary feedback source.

The **exo cable** (Damiao motor) *does* have a spring inserted via key rings (see Case B).d taut, motor angle $\approx$ joint angle. The AS5048A absolute encoder reads the true joint angle directly from the joint axis and is the primary feedback source.

The **exo cable** (Damiao motor) *does* have a spring inserted via key rings (see Case B).

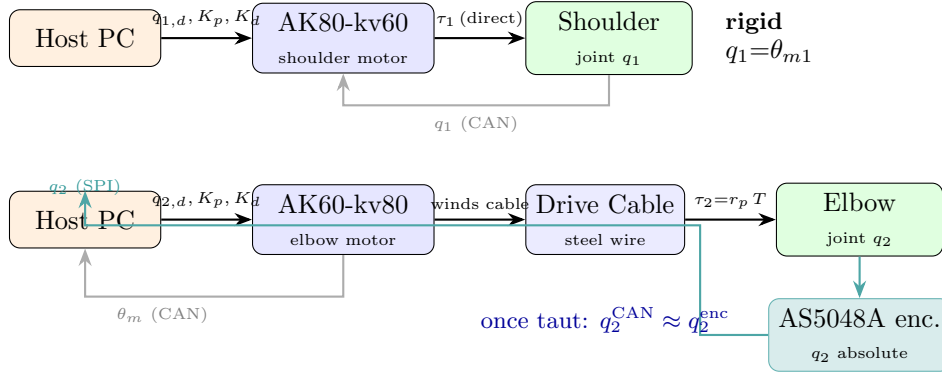


Figure 4: Our arm signal flow. **Shoulder**: rigid direct drive ($q_1 = \theta_{m1}$). **Elbow**: direct steel cable (no spring); once taut, CAN angle $\approx$ encoder angle. The AS5048A reads the true joint angle via SPI and is the primary elbow feedback source.

4.3 Case A: Drive Cable (No Spring) — Manual Home Positioning

This is the current hardware configuration for the elbow drive cable. The steel cable runs from the motor drum, over the 623zz idler bearings, and directly onto the elbow joint drum — no spring in the cable path.

Key advantage: any joint angle can be set as the home position *on every startup*. You simply hold the arm at your desired angle, set the encoder zero, and enable low-tension mode. This is flexible and safe.

Why this works for any control scheme. When you call `set_zero(burn_otp=False)`, you are **redefining the reference frame** for angle measurements. All control laws (CT, LQR, impedance) depend only on *relative* angles and velocities, not on an absolute world reference.

- **Gravity term** $g(q)$: Depends only on angle q relative to home. If you hold at any position and set zero there, gravity is correctly computed relative to that custom reference frame.
- **Inertia** $M(q)$ and **Coriolis** $C(q, \dot{q})$: Purely kinematic; invariant to reference frame shifts.
- **Feedback error** $(q_d - q)$: Still valid; error is measured in your coordinate system.

Therefore, **flexible any-angle home is compatible with CT, LQR, and all other control laws**. It is purely a change of reference frame, not a change in dynamics.

Alternative home-positioning methods. There are three practical ways to set a home position. Table 1 compares them, including which methods **require manual holding** and which are fully automated.

Method	How	Pros / Cons	Persists?	Hold?
A. Manual hold + RAM zero (used in this section)	Hold arm at desired angle, call <code>set_zero(burn_otp=False)</code>	+ Any angle as home, fully flexible + Safe: low-tension from home – Must redo every power-on	RAM only (lost on power-off)	Yes (each startup)
B. OTP burn (optional, documented below)	Same as A, then call <code>set_zero(burn_otp=True)</code> once	+ Auto-zero every power-on + No hold needed after burn – One-time only, irreversible	Permanent (OTP fuse)	Once only
C. Software offset set (no hardware write)	Read raw encoder, subtract a constant: <code>q = enc.read() - HOME_OFFSET</code>	+ No encoder modification + Easily changeable in code – Must store offset externally	In code / config file	Yes (each startup)

Table 1: Comparison of home-positioning methods for the elbow joint.

Recommended method

To skip manual holding altogether:

- **Method B (OTP burn)**: Hold the arm *once today*, burn the zero. Every future startup is fully automatic — no holding ever again. This is the best long-term solution.

Method A (manual hold + RAM zero) is recommended during development: flexible, safe, no hardware commitment. Once home is confirmed, promote to Method B.

Method C: software offset (code example). If you prefer not to call `set_zero()` at all, just store the raw angle at startup and subtract it throughout the session:

Listing 2: Software offset: no encoder write needed.

```

1 with AS5048A(AS5048AConfig()) as enc:
2     # Record raw angle at current position (= your chosen home)
3     HOME_OFFSET = enc.read_angle_rad()
4
5 # ... later in control loop:
6 q2 = enc.read_angle_rad() - HOME_OFFSET # relative to home

```

Two joints — two different home strategies

Joint	Sensor	Home method
Shoulder q_1	CAN only (no encoder)	OTP burn across power cycles
Elbow q_2	CAN + AS5048A encoder	RAM zero gives independence

The procedure below covers **both joints in parallel**. Should shoulder notes: **[ELBOW]**.

Step-by-step procedure (Method A).

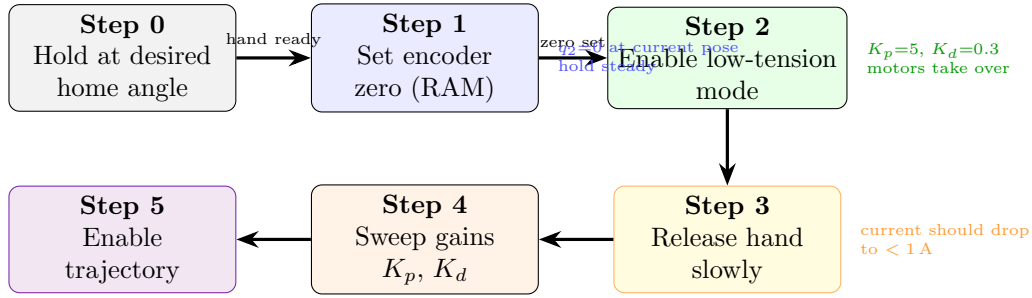


Figure 5: Commissioning flow: hold arm at any desired home angle, then low-tension takeover.

Step 0 — Hold arm at desired home angle. Place the arm at **any angle you want to be “home”** (fully extended, elbow at 90°, flexed, etc.). Support the weight with your hand and keep it steady. The cable will be slack initially — motors are off.

Example. Suppose you want startup home to be: shoulder pointing 30° forward from the base, and elbow bent at 90°. Manually hold the arm in exactly that pose. Nothing is “zero” yet — you are only choosing the physical pose that will later be called $(q_1, q_2) = (0, 0)$.

Listing 3: Hand-position the arm at desired home.

```

1 input("1. Position arm at DESIRED HOME ANGLE.\n")
2     " Support weight with your hand.\n"
3     " Press Enter when ready...\n")

```

Step 1 — Set encoder zero at current position. Read the encoder and set it to zero in RAM (not permanent OTP). This makes the current position “ $q_2 = 0$ ” for this session.

[SHOULDER]: No AS5048A — use the CAN motor angle directly. The motor reports its current output shaft angle; record this as the shoulder home reference. For repeatable startup, burn OTP once (see end of this section).

[ELBOW]: Use the AS5048A absolute encoder. This gives the true joint angle, independent of cable spring deflection.

Example. At the held pose from Step 0, the shoulder CAN may read $q_1 = 27.4$ and the elbow AS5048A may read $q_2 = 91.2$. After this step, the controller will treat those same physical angles as the home reference: shoulder target stored as `q1_start = 27.4°`, and elbow encoder reset so it now reports approximately 0.0 at that same pose.

Listing 4: Set zeros for both joints.

```

1 # --- SHOULDER: CAN-only zero (no secondary encoder) ---
2 q1_start, _, _ = shoulder.read_state()["shoulder"]
3 print(f"Shoulder CAN zero: {np.rad2deg(q1_start):.1f}deg")
4 # q1_start is the motor output-shaft angle at home;
5 # controller uses relative error (q_d - q1_start)
6 # Recommend burning this to OTP once confirmed (see end of section)
7
8 # --- ELBOW: AS5048A absolute encoder zero ---
9 with AS5048A(AS5048AConfig()) as enc:
10     enc.set_zero(burn_otp=False) # RAM only, temporary
11     q2_zero = enc.read_angle_rad()
12     print(f"Elbow encoder zero set. Current angle: {np.rad2deg(q2_zero):.1f}deg")

```

Step 2 — Enable low-tension mode. Power on both motors with **very low** K_p (5). The motors will start holding the arm at the position you’re still supporting with your hand. The cable begins to wind up but with minimal force.

Example. If you are still holding the chosen home pose, commanding `shoulder = q1_start` and `elbow = 0.0` with low gains means the motors take over gently. You should feel only a light holding force, not a sharp pull.

Listing 5: Motors take over with low tension.

```
1 print("2. Motors powering on (low tension)...")
2 shoulder.write("goal_position", {"shoulder": q1_start}, kp=5.0, kd=0.3)
3 elbow.write("goal_position", {"elbow": 0.0}, kp=5.0, kd=0.3) # 0 = home
4 time.sleep(0.3)
```

Step 3 — Slowly release your hand. Gradually let go over 1–2 seconds. Watch the motor current on the console. As you release, motor current should **drop toward zero** (cable slack remaining, both cables neutral). If current rises steeply, you may not be at a true zero position — arm is not at mechanical equilibrium.

Example. Good release case: shoulder current settles near 0.2–0.4 A and elbow current settles near 0.3 A, with the arm staying in place. Bad release case: elbow current rises to 2.5 A and the joint creeps a few degrees, which means the elbow zero was set away from the neutral cable position — redo from Step 0.

△ Important

Check motor current during release — both joints.

- **[SHOULDER]** — direct-drive, no cable spring: current should drop to near 0 A immediately on release. A steady current > 1 A means the shoulder home angle was set while the motor shaft was not at its cable equilibrium.
- **[ELBOW]** — antagonistic cable SEA:
 Current < 0.5 A after release: both cables neutral.
 Current 1–2 A: arm is off the neutral cable position; acceptable but not ideal.
 Current > 2 A: wrong zero angle — redo from Step 0.

Listing 6: Monitor current while releasing.

```
1 print("3. Release your hand slowly. Watch current below:")
2 for i in range(10):
3     _, _, i_a = elbow.read_state()["elbow"]
4     q2_enc = enc.read_angle_rad()
5     print(f" Current: {i_a:.2f} A | Encoder: {np.rad2deg(q2_enc):.1f}")
6     time.sleep(0.3)
```

Step 4 — Sweep gains. Once the arm is stable on the motors, increase K_p gradually in steps to reach operating gains. Monitor for oscillation. Target range: $K_p = 30$ –50, $K_d = 0.8$ –1.5.

Example. You might test $K_p = 5, 10, 20, 30, 40$ while keeping the arm at home. If the arm stays quiet up to $K_p = 30$ but starts buzzing or overshooting at $K_p = 40$, then $K_p = 30$ with slightly larger K_d is a safer operating choice.

Listing 7: Ramp up gains from 5 to 40.

```
1 for kp in [5, 10, 15, 20, 30, 40]:
2     shoulder.write("goal_position", {"shoulder": q1_start}, kp=kp, kd=0.5)
3     elbow.write("goal_position", {"elbow": 0.0}, kp=kp, kd=0.5)
4     print(f"Testing Kp={kp}...")
5     time.sleep(1.0)
6 print(" Gains stable. Ready for trajectory.")
```

Step 5 — Enable trajectory. Send your first trajectory command. The encoder zero you set remains valid for the entire session.

Example. If the chosen home pose is the arm's comfortable resting pose, then a command like $q_1^d = q_{1,\text{start}} + 0.3 \sin(0.3t)$ and $q_2^d = 0.0 + 0.2 \sin(0.5t)$ means the shoulder oscillates around its stored startup angle, while the elbow oscillates around the newly defined home angle $q_2 = 0$.

Listing 8: Send trajectory (example: sine wave).

```

1 t0 = time.monotonic()
2 while True:
3     t = time.monotonic() - t0
4     # Oscillate around the home position
5     q1_d = q1_start + 0.3 * np.sin(0.3*t)
6     q2_d = 0.0 + 0.2 * np.sin(0.5*t) # 0 is home
7
8     shoulder.write("goal_position", {"shoulder": q1_d}, kp=40.0, kd=1.2)
9     elbow.write("goal_position", {"elbow": q2_d}, kp=30.0, kd=1.0)
10    time.sleep(0.01)

```

Optional: Burn elbow encoder zero to OTP (permanent). If you want the **elbow** home to survive power cycles, burn the AS5048A zero permanently after you've verified the arm is stable:

Listing 9: Permanently burn elbow OTP zero (one-time only).

```

1 print("Burning ELBOW encoder zero to OTP (PERMANENT, IRREVERSIBLE)...")
2 with AS5048A(AS5048AConfig()) as enc:
3     enc.set_zero(burn_otp=True)
4 print("OTP burned. This angle will be zero on every future power-up.")

```

Shoulder: burn CAN zero to OTP (recommended once). The shoulder has **no secondary encoder** — it relies entirely on the CAN motor angle. Because the CAN zero is lost on power-off unless burned, burn OTP once after confirming the mechanical home.

When to burn: after Step 3 confirms low current (stable home), and before running any trajectory.

Listing 10: Burn shoulder home to OTP — run once only.

```

1 # SHOULDER: burn current CAN angle as permanent zero
2 # ONLY run this after Step 3 confirms low current (correct home)
3 print("Burning SHOULDER zero to OTP (PERMANENT, IRREVERSIBLE)...")
4 shoulder.encoder.set_zero(burn_otp=True)
5 # Verify: CAN should now report ~0.0 deg
6 q1_check, _, _ = shoulder.read_state()["shoulder"]
7 print(f"Shoulder CAN after burn: {np.rad2deg(q1_check):.1f}deg (expect ~0.0)")
8 print("Done. Shoulder home will load automatically on every future power-on.")

```

Shoulder vs Elbow: OTP lifetime rule

- **Shoulder:** OTP burn is **essential** for reproducibility (no absolute encoder to re-zero each session). Burn once when home is confirmed.
- **Elbow:** OTP burn is **optional**. The AS5048A lets you re-zero in RAM every session with no loss of accuracy. Only burn OTP once your home angle is finalized.
- **Both:** OTP fuses are one-time only. Use `burn_otp=False` during development.

How burning works. When you call `set_zero(burn_otp=True)`, the AS5048A records the **current magnetic angle to permanent fuses**. The magnet is bolted to the joint — it is purely passive (no power needed).

On every future power-on:

1. The sensor wakes up and reads the magnet position (always accurate).
2. It subtracts the burned OTP offset automatically.
3. Output = (current magnet angle) – (burned OTP offset).

Safe to move the joint while powered off. The magnet is the source of truth — it does **not** require power. If you manually rotate the joint while the sensor is off:

- The magnet rotates with the joint (mechanically coupled).
- When you power on, the sensor reads the magnet’s **new position**.
- Output is still correct because it’s always relative to the burned offset.

Example: If OTP burned at 0° and you manually move the joint to 60° while powered off, the sensor powers on and reads: $60 - 0 = 60$

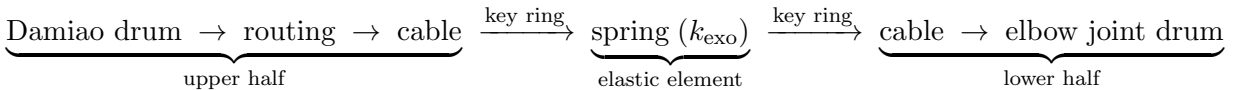
△ Important

Each encoder can only burn OTP once — this is irreversible. Only use ‘burn_otp=True’ when you are certain of your home position. For experimentation, use ‘burn_otp=False’ (RAM only, cleared on power-off).

Safe to physically move the arm when powered off. The magnet is passive — no homing sequence needed after power-on.

4.4 Case B: Exo Cable (Spring via Key Rings) — Damiao Motor

This applies to the exo cable only. The Damiao motor drives a steel cable that has a **coil spring inserted mid-span via two key rings** (quick-links):



The spring makes the exo cable compliant: torque is proportional to deflection $\delta_e = r_{\text{exo}}(\theta_{me} - q_2)$. The bring-up is more involved because:

- The cable may be **slack** at power-on.
- Damiao CAN angle $\theta_{me} \neq q_2$ (spring deflection unknown at power-on).
- A stiff command before the cable is taut can snap or tangle it.

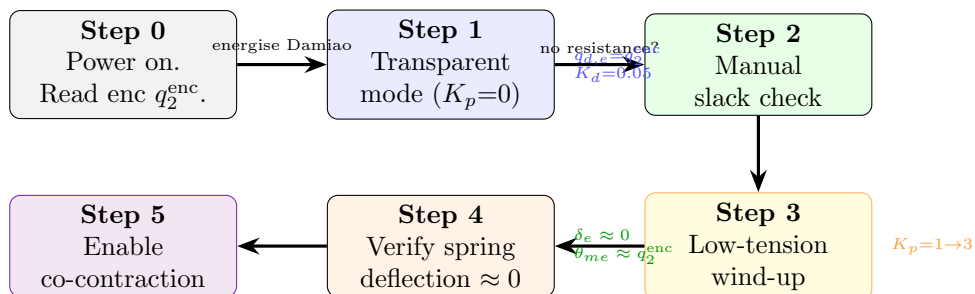


Figure 6: Commissioning flow for the exo cable (Damiao, spring via key rings).

Step-by-step procedure.

Step 0 — Power on, read elbow encoder. Read the elbow angle before energising the Damiao. This angle will be the transparent-mode target.

Listing 11: Read elbow angle before energising Damiao.

```
1 q2_enc = enc.read_angle_rad() # absolute, no homing
2 print(f"Elbow at power-on: {np.rad2deg(q2_enc):.1f} deg")
```

Step 1 — Set Damiao to transparent mode. $K_p = 0$, light damping only. The motor tracks q_2^{enc} so cable extension is zero: $\delta_e \approx 0$, no force on the joint.

Listing 12: Damiao transparent mode from power-on.

```
1 exo.write("mit_command", {"exo": q2_enc},
2         kp=0.0, kd=0.05, tau_ff=0.0) # zero tension
```

Step 2 — Manual slack check. With the Damiao in transparent mode, **manually flex the elbow slowly**. The exo cable should offer *no resistance*. If you feel tension, check the key-ring attachment direction.

△ Important

Check key-ring and spring orientation. The spring must extend when the cable is pulled (not compressed). A reversed connection means the spring is buckled rather than stretched — the cable will feel stiff in the wrong direction and may tangle.

Step 3 — Low-tension wind-up. Slowly increase K_p to gently take up any cable slack, still targeting $q_{d,e} = q_2^{\text{enc}}$.

Listing 13: Low-tension wind-up for exo cable.

```
1 for kp in [1.0, 2.0, 3.0]: # ramp Kp in steps
2     exo.write("mit_command", {"exo": enc.read_angle_rad()},
3             kp=kp, kd=0.3, tau_ff=0.0)
4     time.sleep(0.3)
```

Step 4 — Verify spring deflection near zero. At rest the Damiao CAN angle should satisfy $\theta_{me} \approx q_2^{\text{enc}}$. Their difference equals the spring deflection δ_e/r_{exo} .

Listing 14: Check spring deflection at rest.

```
1 theta_me, _, _ = exo.read_state()["exo"]
2 q2_enc = enc.read_angle_rad()
3 delta_e = 0.04775 * (theta_me - q2_enc) # deflection [m]
4 print(f"Spring deflection: {delta_e*1000:.1f} mm (expect ~0 at rest)")
```

Step 5 — Enable co-contraction. Activate with a small offset $\Delta\theta$ to pre-tension the spring.

Listing 15: Enable exo co-contraction.

```
1 DELTA_THETA = np.deg2rad(5.0) # 5 deg -> gentle pre-tension
2 exo.write("mit_command",
3         {"exo": enc.read_angle_rad() + DELTA_THETA},
4         kp=5.0, kd=0.3, tau_ff=0.0)
```


△ **Important**

Never activate co-contraction before completing Steps 1–4. A large $\Delta\theta$ with stiff gains on a slack cable will snap the spring-cable assembly onto the elbow joint drum, jerking the elbow joint.

4.5 Why Cable Compliance Changes Everything vs. Traditional Control

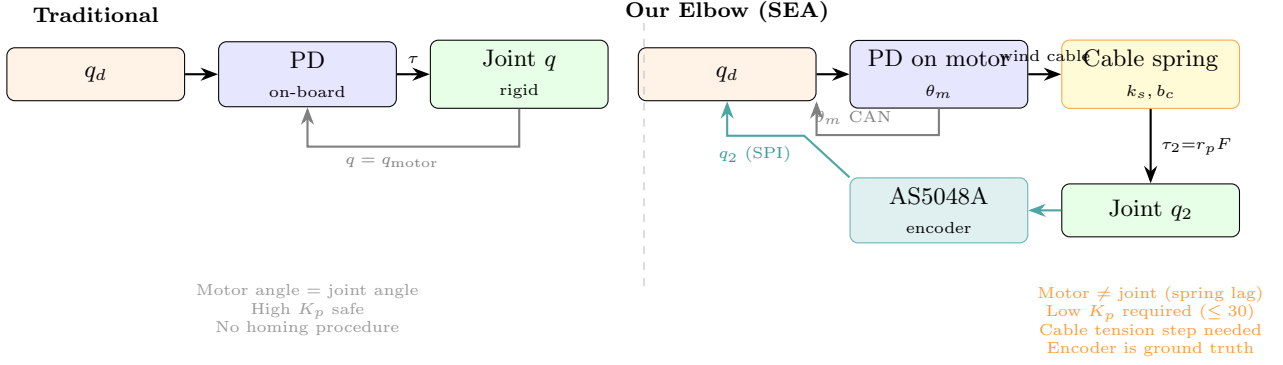


Figure 7: Side-by-side comparison: traditional rigid joint vs. our SEA cable elbow.

Table 2: Traditional robot vs. our SEA cable elbow: key differences.

Property	Traditional (rigid)	Ours (SEA cable)
Transmission	Gearbox, rigid	Cable + spring (k_s, b_c)
Motor = joint?	Yes, always	No, differ by δ/r_p
Feedback to PD	Joint angle (same wire)	Motor angle θ_m only
Ground-truth angle	Motor encoder	AS5048A absolute encoder
Max safe K_p	100+	≤ 30 (cable resonance)
Power-on sequence	Power $\rightarrow$ home	Slack check $\rightarrow$ low tension $\rightarrow$ ramp $\rightarrow$ home
Spring lag	None	Yes; grows with torque load
Cable slack risk	None	Yes; must be taken up slowly
Force sensing	External F/T sensor	Spring deflection δ/r_p is implicit force

5 Phase 1 — Manipulator Only (MIT PD, No Model)

★ **Phase 1:**

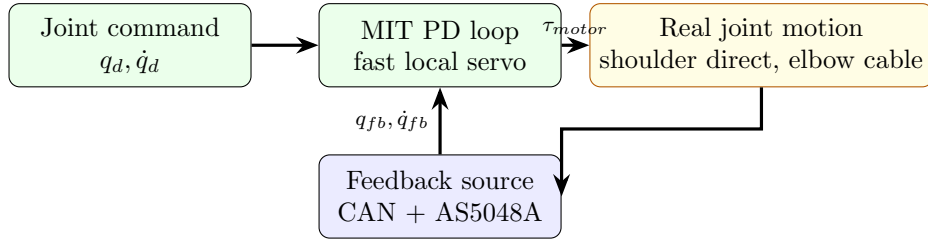
Motivation and when to use (arm only) At the start of hardware commissioning the exact link masses and centres of mass are **unknown**. Building a gravity-compensation or computed-torque controller would require prior identification of these parameters.

Run this first — shoulder + elbow only, no exo. Use each motor's own on-board MIT PD loop as the controller. The on-board loop runs at the motor's internal control rate (≈ 5 kHz), far faster than the host CAN loop (100 Hz), providing inherent stability even under CAN jitter. The AS5048A encoder feeds back the true elbow joint angle (not the motor-side angle).

Expected limitation: friction, joint stiction, and cable pre-tension are uncompensated (gravity is negligible for this horizontal-plane arm). Steady-state error is proportional to friction torque / K_p . This is acceptable for initial motion verification, encoder synchronisation, and gain sweeps before the full model is available.

Lecture Focus

- **Teaching point 1:** Phase 1 is the minimal controller that gets the real robot moving safely.
- **Teaching point 2:** No full dynamic model is needed yet; the motor drives its own local PD loop.
- **Teaching point 3:** The elbow is harder than the shoulder because the motor sees cable-side motion, not the true joint angle.
- **Teaching point 4:** The purpose of Phase 1 is validation: motor direction, encoder alignment, gain tuning, and basic tracking.



Key lecture contrast: for the shoulder, motor feedback is close to joint feedback; for the elbow, the cable-spring path makes the sensed and actual quantities more subtle.

Figure 8: Phase 1 teaching visual: what the MIT PD loop is actually doing on the real arm.

5.1 MIT on-board PD

The CubeMars motor implements a **proportional-derivative (PD) controller** *on the motor controller chip itself*. Each CAN frame specifies desired angle and gains (K_p, K_d); the motor runs a closed-loop servo without host feedback. The control law is:

$$\tau_{\text{motor}} = K_p(q_d - q_{\text{fb}}) + K_d(\dot{q}_d - \dot{q}_{\text{fb}}) + \tau_{\text{ff}}, \quad (17)$$

with $K_p \in [0, 500]$ Nm/rad and $K_d \in [0, 5]$ Nm-s/rad embedded in CAN. Setting $\tau_{\text{ff}}=0$ gives a pure on-board spring-damper.

5.1.1 Shoulder vs. Elbow: What MIT Sees

The **critical difference** is what q_{fb} represents:

- **Shoulder (AK80-kv60, direct drive):** Motor shaft rigidly bolted to joint axis. MIT feedback is the *joint angle* q_1 directly. Torque appears at the joint immediately.
- **Elbow (AK60-kv80, cable SEA):** Motor shaft *is* the DrivePulley. MIT feedback is the *motor-side angle* θ_m (not the joint angle q_2). Torque must first deform the cable spring; the joint lags behind.

5.1.2 Case 1: Direct Cable (No Spring) — Thought Experiment

Imagine the elbow drive used a steel cable **without the spring inserted** (same hardware, just remove the spring from the key-ring chain and use a solid link instead). Once the cable is wound taut:

$$\text{(Hypothetical) } q_2 \equiv \theta_m. \quad (18)$$

With rigid coupling, MIT PD would act *directly* on the joint:

$$\tau_{\text{motor}} = K_p(q_{2,d} - q_2) + K_d(\dot{q}_{2,d} - \dot{q}_2) \quad (19)$$

$$= K_p(q_{2,d} - \theta_m) + K_d(\dot{q}_{2,d} - \dot{\theta}_m) \quad (20)$$

Joint and motor would move together ($q_2 = \theta_m$), equivalent to commanding the joint directly. Response would be fast and immediate. MIT could use high gains.

5.1.3 Case 2: Elastic Cable (With Spring) — Actual System

Without load cell: Our elbow uses a **cable spring** with stiffness k_s . We use the **CubeMars CAN output-shaft angle** convention throughout this subsection: θ_m is the motor *output shaft* angle reported over CAN, so there is **no division by N in the kinematics**. The base idea is a simple torque balance:

- $\tau_{m,\text{cmd}}$: torque commanded by the MIT controller at the motor output shaft.
- τ_{spring} : reaction torque generated by spring stretch in the cable path.
- τ_2 : actual torque delivered to elbow joint 2.

$$\tau_{m,\text{cmd}} - \tau_{\text{spring}} = J_{m,\text{out}} \ddot{\theta}_m, \quad \tau_2 = \tau_{\text{spring}}. \quad (21)$$

That is, the motor output-shaft torque first accelerates the motor inertia and the remaining transmitted torque appears at the joint through the spring. The spring torque comes from cable deflection:

$$\delta = r_p(\theta_m - q_2), \quad F_{\text{cable}} = k_s \delta + b_c \dot{\delta}, \quad \tau_{\text{spring}} = \tau_2 = r_p F_{\text{cable}}. \quad (22)$$

With load cell: If an in-line **load cell** is added to the cable, then the structure of the SEA equations does not change, but the force term becomes *measured* rather than purely inferred from the spring model. In that case one may write

$$F_{\text{cable}} \approx F_{\text{meas}}, \quad \tau_{\text{spring}} = \tau_2 = r_p F_{\text{meas}}. \quad (23)$$

The spring relation $F_{\text{cable}} = k_s \delta + b_c \dot{\delta}$ is still useful as an internal model or consistency check, but it is no longer the only source of force information. This improves observability of preload, slack, friction, and hysteresis. However, it does *not* create a new direct actuator input: the controller still commands only $\tau_{m,\text{cmd}}$, while δ , F_{cable} , and τ_2 remain internal states or measured outputs of the coupled motor-spring-joint system.

Substituting this spring torque into the motor and joint equations gives the full 2nd-order dynamics:

$$J_{m,\text{out}} \ddot{\theta}_m = \tau_{m,\text{cmd}} - r_p F_{\text{cable}} \quad (\text{motor output shaft}) \quad (24)$$

$$M(q_2) \ddot{q}_2 + h(q_2, \dot{q}_2) = r_p F_{\text{cable}} \quad (\text{joint}) \quad (25)$$

where the MIT controller generates

$$\tau_{m,\text{cmd}} = K_p(q_{2,d} - \theta_m) + K_d(\dot{q}_{2,d} - \dot{\theta}_m). \quad (26)$$

So the CubeMars command flows to the elbow joint through the SEA in the following order:

$$\tau_{m,\text{cmd}} \longrightarrow \ddot{\theta}_m, \theta_m \longrightarrow \delta = r_p(\theta_m - q_2) \longrightarrow F_{\text{cable}} \longrightarrow \tau_2 = r_p F_{\text{cable}} \longrightarrow q_2. \quad (27)$$

Here $J_{m,\text{out}}$ is the motor inertia reflected to the output shaft. Equivalently, one may write the same equation in rotor-side variables using $\theta_r = N\theta_m$, but then the spring term and torque

command both carry factors of $1/N$. We avoid that mixed convention here because the hardware gives θ_m directly via CAN.

MIT PD acts on motor position, not joint position. The spring creates *time lag*. If K_p is too high, the motor output shaft overshoots, stretches the spring, and stores elastic energy before the joint catches up. The spring then releases that energy back into the motor-joint pair—causing **cable resonance** (underdamped oscillation).

✓ Key Result

Why the spring limits gain: MIT closes the loop on motor position θ_m , not joint position q_2 . High K_p forces the motor output shaft to accelerate aggressively, straining the spring. The spring then resists and feeds energy back into the motor-joint pair, oscillating the system around equilibrium. The maximum stable $K_p \approx 0.5 k_s r_p^2$ (order-of-magnitude). We use $K_p \leq 30$ to stay well clear of this threshold.

Gain selection without model.

- Start conservative: $K_p=20$, $K_d=0.5$ for both joints.
- Increase K_p in steps of 5 until joint tracks a step command.
- Increase K_d until oscillations are damped (typically $K_d \approx 0.05\sqrt{K_p}$ is a safe starting rule).
- Shoulder (AK80-kv60, **direct drive**): higher effective inertia; can tolerate K_p up to 60 before feeling stiff.
- Elbow (AK60-kv60, **cable SEA**): the on-board MIT PD command goes to the *motor* drum. The MIT K_p acts on motor position θ_m , not the joint. Keep $K_p \leq 30$ to avoid exciting cable resonance (the spring k_s limits torque bandwidth).

Listing 16: Phase 1 gain selection and MIT PD for both joints.

```

1 # Phase-1 gains (no model, tune empirically)
2 KP_SHOULDER, KD_SHOULDER = 40.0, 1.2 # AK80-kv60 shoulder
3 KP_ELBOW, KD_ELBOW = 25.0, 0.8 # AK60-kv80 elbow (cable -> lower Kp)
4
5 # On-board MIT PD (tau_ff = 0): each motor closes its own loop
6 shoulder.write("goal_position", {"shoulder": q_d[0]},
7               kp=KP_SHOULDER, kd=KD_SHOULDER)
8 elbow.write("goal_position", {"elbow": q_d[1]},
9            kp=KP_ELBOW, kd=KD_ELBOW)

```

5.2 AS5048A encoder as elbow angle source

The AS5048A is a **14-bit absolute magnetic encoder**. Register 0x3FFF returns the angle already corrected by the chip's OTP zero-offset register, so `read_angle_rad()` gives the correct joint angle on *every* power-up with no host-side software compensation.

$$q_2^{\text{enc}} = \text{counts} \cdot \frac{2\pi}{2^{14}} \quad (\text{zero already in chip OTP}). \quad (28)$$

Listing 17: AS5048A — absolute angle read (no offset arithmetic needed).

```

1 from encoder_config import AS5048AConfig
2 from as5048a import AS5048A
3
4 enc = AS5048A(AS5048AConfig(bus=0, device=0, max_hz=1_000_000))
5

```

```

6 with enc:
7     q2 = enc.read_angle_rad() # [0, 2*pi), absolute, zero-corrected by chip
8     # That's it. No offset subtraction required after OTP is burned.

```

One-time OTP commissioning (done once, irreversible). Before running any control, mount the encoder and move the elbow to the mechanical zero ($q_2=0$, arm fully extended), then burn the zero permanently:

Listing 18: One-time OTP zero burn. Do NOT repeat – fuses are blown permanently.

```

1 with AS5048A(AS5048AConfig()) as enc:
2     raw = enc.set_zero(burn_otp=True) # <<< ONE TIME ONLY
3     print(f"Zero burned. Mechanical angle latched: {raw} counts")

```

After this, `read_angle_rad()` on every subsequent power-up returns the joint angle directly. There is no need to move to a home position or call any zeroing function.

△ Important

`set_zero(burn_otp=False)` writes the zero offset to **chip RAM** only and is *lost on power cycle*. Always use `burn_otp=True` for permanent commissioning. If the OTP is already burned, do *not* call `set_zero` again — the fuse can only be programmed once.

Encoder vs. CAN comparison check. At rest (no cable tension), $|q_2^{\text{enc}} - q_2^{\text{CAN}}|$ should be $< 0.5^\circ$. Larger offset at rest indicates a wiring or zero-calibration error. Offset growing with applied torque is normal (cable spring extension) and confirms the SEA model is relevant.

5.3 Complete Phase 1 control loop (Arm Only)

Listing 19: Phase 1 arm-only loop: MIT PD for shoulder + elbow, no model.

```

1 import time
2 import numpy as np
3 from cubemars.ak_v1.ak_v1_bus import CubeMarsAkV1Bench
4 from cubemars.ak_v3.ak_v3_bus import CubeMarsAkV3Bench
5 from as5048a import AS5048A
6 from motor_config import (
7     CubeMarsAkV1BusConfig, CubeMarsAkV1MotorConfig,
8     CubeMarsAkV3BusConfig, CubeMarsMotorConfig,
9 )
10 from encoder_config import AS5048AConfig
11
12 # ---- Hardware objects (arm only: shoulder + elbow) ----
13 shoulder = CubeMarsAkV1Bench(channel="can1", can_id=0x01,
14                             model="AK80-8", name="shoulder")
15 elbow = CubeMarsAkV3Bench(channel="can1", can_id=0x02,
16                            model="AK60-6", name="elbow")
17 enc = AS5048A(AS5048AConfig(bus=0, device=0))
18
19 # ---- Phase-1 PD gains (no model) ----
20 KP_SHOULDER, KD_SHOULDER = 40.0, 1.2
21 KP_ELBOW, KD_ELBOW = 25.0, 0.8
22
23 # ---- Slow sinusoid for initial motion verification ----
24 def traj_phase1(t):
25     q_d = np.array([0.3*np.sin(0.3*t), 0.2*np.sin(0.5*t)])
26     dq_d = np.array([0.3*0.3*np.cos(0.3*t), 0.2*0.5*np.cos(0.5*t)])
27     return q_d, dq_d

```

```

28
29 DT = 0.01 # 100 Hz
30
31 with shoulder, elbow, enc:
32     shoulder.enable()
33     elbow.enable()
34     t0 = time.monotonic()
35     try:
36         while True:
37             t = time.monotonic() - t0
38             # --- Read state (encoder gives joint angle directly) ---
39             q2_enc = enc.read_angle_rad() # AS5048A absolute
40             q1, dq1, _ = shoulder.read_state()["shoulder"] # CAN (direct drive)
41             _, dq2, _ = elbow.read_state()["elbow"] # CAN velocity only
42
43             # --- Reference ---
44             q_d, dqd = traj_phase1(t)
45
46             # --- MIT on-board PD (tau_ff = 0) ---
47             shoulder.write("goal_position", {"shoulder": q_d[0]},
48                           kp=KP_SHOULDER, kd=KD_SHOULDER)
49             elbow.write("goal_position", {"elbow": q_d[1]},
50                        kp=KP_ELBOW, kd=KD_ELBOW)
51
52             time.sleep(max(0.0, DT - (time.monotonic() - t0 - t)))
53
54     except KeyboardInterrupt:
55         print("Stopping -- zeroing all torques")
56     finally:
57         shoulder.write("goal_torque", {"shoulder": 0.0})
58         elbow.write("goal_torque", {"elbow": 0.0})

```

5.4 Phase 1 commissioning checklist

1. **Encoder zero (OTP) — done once only.** Move elbow to mechanical zero ($q_2=0$), run `enc.set_zero(burn_otp=True)`. Never repeat. Afterwards `read_angle_rad()` always returns the correct joint angle with no further setup.
2. **Direction check:** command $q_1 = +0.1$ rad; verify shoulder moves in positive direction. Flip sign if reversed.
3. **Encoder agree:** compare q_2^{CAN} with q_2^{enc} at rest ($< 0.5^\circ$ expected). Under load the CAN angle will lag the encoder by δ/r_p (spring deflection) — this is expected and correct. The encoder is ground truth; use it as q_2 in the controller.
4. **Gain sweep:** run slow-sine trajectory; increase K_p in steps of 5 until tracking error $< 5^\circ$.
5. **E-stop test:** Ctrl+C — both arm motors coast to zero within one loop tick.

6 Phase 2 — Manipulator Only (Computed Torque)

★ Phase 2:

When to use (arm only) After link masses (m_1, m_2) and CoM distances (l_{c1}, l_{c2}) are measured, the Computed-Torque (CT) law inverts the full nonlinear dynamics, cancelling inertia and Coriolis coupling exactly. Gravity is negligible (arm moves in the horizontal XY plane; $\mathbf{g}(\mathbf{q}) = \mathbf{0}$). The closed-loop error dynamics become a linear second-order system at the design bandwidth ω_n .

Run this with shoulder + elbow only; no exo yet.

Lecture Focus

- **Teaching point 1:** Phase 2 is not a different robot; it is the same robot with a better model.
- **Teaching point 2:** The model parameters come from measurement plus CAD, not from guessing the final matrices directly.
- **Teaching point 3:** CT improves tracking by cancelling inertia and coupling before the PD terms act.
- **Teaching point 4:** SEA compensation and inner damping are practical fixes for what the ideal CT derivation does not capture.

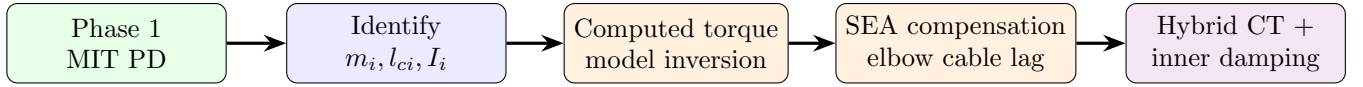


Figure 9: Controller progression to discuss in lecture: each stage solves a real limitation exposed by the previous one.

6.1 Parameter identification workflow

Parameter	Method	Units
m_1, m_2	Weigh links (with motors attached) on a scale	kg
l_{c1}, l_{c2}	Balance-point measurement or CAD export	m
I_1, I_2 (moments of inertia)	Uniform rod: $I_i = m_i L_i^2 / 3$; or CAD	kg·m ²
r_p	Calliper measurement of drive pulley OD	m
k_s	Static force-extension test on cable	N/m

Quick swing test: lock the shoulder, give the elbow a small impulse and measure the free-oscillation frequency ω_{osc} (using the AS5048A encoder). Then $I_2 = k_{\text{spring}} / \omega_{\text{osc}}^2$ where k_{spring} is a known torsional restoring stiffness applied via the exo cable. (The standard pendulum formula $m_2 l_{c2} = I_2 \omega^2 / g$ does **not** apply here: gravity acts in $-Z$, perpendicular to the plane, and contributes no restoring torque.)

6.2 Why the CT law has this form

The goal is to make the actual joint motion $\mathbf{q}(t)$ follow a commanded trajectory $\mathbf{q}_d(t)$ as closely as possible.

Start by defining the tracking error:

$$\mathbf{q}_e = \mathbf{q}_d - \mathbf{q}, \quad \dot{\mathbf{q}}_e = \dot{\mathbf{q}}_d - \dot{\mathbf{q}}, \quad \ddot{\mathbf{q}}_e = \ddot{\mathbf{q}}_d - \ddot{\mathbf{q}}. \quad (29)$$

If we could directly command joint acceleration, we would ask for the error to obey a simple linear second-order system,

$$\ddot{\mathbf{q}}_e + \mathbf{K}_d \dot{\mathbf{q}}_e + \mathbf{K}_p \mathbf{q}_e = \mathbf{0}. \quad (30)$$

We choose a second-order error model because the robot itself is a second-order mechanical system: torque affects acceleration, acceleration changes velocity, and velocity changes position. This form is also the simplest one that lets us shape both tracking speed and damping through $\mathbf{K}_p$ and $\mathbf{K}_d$. Rearranging (30) gives the acceleration we would like the robot to have:

$$\ddot{\mathbf{q}} = \ddot{\mathbf{q}}_d + \mathbf{K}_d \dot{\mathbf{q}}_e + \mathbf{K}_p \mathbf{q}_e. \quad (31)$$

This motivates the definition of the virtual input

$$\mathbf{v} = \ddot{\mathbf{q}}_d + \mathbf{K}_d \dot{\mathbf{q}}_e + \mathbf{K}_p \mathbf{q}_e. \quad (32)$$

What does “virtual input” mean? The physical input to the robot is torque τ , not acceleration. The vector $\mathbf{v}$ is therefore a design quantity: it is the acceleration behavior we want the joints to have. The role of CT is to convert this desired acceleration behavior into the torque required by the real nonlinear robot.

Small one-joint example. Suppose we look only at the elbow joint. Assume the commanded motion is to hold position, so $q_{d,2} = 0.5 \text{ rad}$, $\dot{q}_{d,2} = 0$, $\ddot{q}_{d,2} = 0$. If the actual elbow state is $q_2 = 0.4 \text{ rad}$ and $\dot{q}_2 = -0.2 \text{ rad/s}$, then

$$e_2 = q_{d,2} - q_2 = 0.1, \quad \dot{e}_2 = \dot{q}_{d,2} - \dot{q}_2 = 0.2. \quad (33)$$

With $K_{p,2} = 100$ and $K_{d,2} = 20$, the corresponding virtual input is

$$v_2 = \ddot{q}_{d,2} + K_{d,2}\dot{e}_2 + K_{p,2}e_2 = 0 + 20(0.2) + 100(0.1) = 14 \text{ rad/s}^2. \quad (34)$$

So the controller would like the elbow to accelerate in the positive direction at 14 rad/s^2 .

Now introduce the *true* manipulator dynamics:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}. \quad (35)$$

For this arm, motion is in the horizontal XY plane, so gravity produces no joint torque and $\mathbf{g}(\mathbf{q}) = \mathbf{0}$. The main nonlinear terms are therefore the inertia matrix $\mathbf{M}(\mathbf{q})$ and the Coriolis term $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$.

In practice, we do not know the exact model. We instead use a predicted model from CAD, measurements, and identification:

$$\tilde{\mathbf{M}}(\mathbf{q}), \quad \tilde{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}}), \quad \tilde{\mathbf{g}}(\mathbf{q}). \quad (36)$$

The practical CT law therefore commands torque using the predicted model:

$$\boldsymbol{\tau} = \tilde{\mathbf{M}}(\mathbf{q})\mathbf{v} + \tilde{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \tilde{\mathbf{g}}(\mathbf{q}). \quad (37)$$

This is an *approximate cancellation* law: it tries to cancel the plant’s nonlinear terms using $\tilde{\mathbf{M}}, \tilde{\mathbf{C}}, \tilde{\mathbf{g}}$, even though these do not perfectly match the true dynamics.

Substituting (37) into (35) gives

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \tilde{\mathbf{M}}(\mathbf{q})\mathbf{v} + \tilde{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \tilde{\mathbf{g}}(\mathbf{q}). \quad (38)$$

or, after rearranging,

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \tilde{\mathbf{M}}(\mathbf{q})\mathbf{v} + (\tilde{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}))\dot{\mathbf{q}} + (\tilde{\mathbf{g}}(\mathbf{q}) - \mathbf{g}(\mathbf{q})). \quad (39)$$

This equation shows the practical case clearly: when the predicted and true models do not match, cancellation is incomplete. CT still helps, but the remaining mismatch terms act like disturbances.

Only in the ideal matched-model case,

$$\tilde{\mathbf{M}}(\mathbf{q}) = \mathbf{M}(\mathbf{q}), \quad \tilde{\mathbf{C}}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}), \quad \tilde{\mathbf{g}}(\mathbf{q}) = \mathbf{g}(\mathbf{q}), \quad (40)$$

does the cancellation become exact. Then (39) reduces to

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})\mathbf{v} \implies \ddot{\mathbf{q}} = \mathbf{v}, \quad (41)$$

assuming $\mathbf{M}(\mathbf{q})$ is invertible. In that ideal case the nonlinear robot is converted into a set of double integrators.

Substituting (32) into (41) gives

$$\ddot{\mathbf{q}} = \ddot{\mathbf{q}}_d + \mathbf{K}_d\dot{\mathbf{q}}_e + \mathbf{K}_p\mathbf{q}_e. \quad (42)$$

Using $\ddot{\mathbf{q}}_e = \ddot{\mathbf{q}}_d - \ddot{\mathbf{q}}$ then yields

$$\ddot{\mathbf{q}}_e + \mathbf{K}_d\dot{\mathbf{q}}_e + \mathbf{K}_p\mathbf{q}_e = \mathbf{0}. \quad (43)$$

So CT is designed in two steps: first choose the acceleration behavior needed for tracking, and then use the model to realize that behavior through torque. In the real robot this cancellation is only approximate, which is why good parameter identification matters and why very large ω_n can amplify modelling errors and excite the SEA compliance at the elbow.

6.3 Computed Torque Control law

$$\tau = M(q)[\ddot{q}_d + K_d \dot{e} + K_p e] + \underbrace{C(q, \dot{q}) \dot{q} + g(q)}_{=0} \quad (44)$$

where $e = q_d - q$, $\dot{e} = \dot{q}_d - \dot{q}$, and $g(q) = 0$ because the arm moves in the horizontal XY plane (gravity acts in $-Z$, perpendicular to the plane of motion). Substituting into (3) gives linear error dynamics: $\ddot{q}_e + K_d \dot{q}_e + K_p q_e = 0$. Choose $K_p = \omega_n^2 \mathbf{I}$, $K_d = 2\omega_n \mathbf{I}$ (critically damped at bandwidth ω_n).

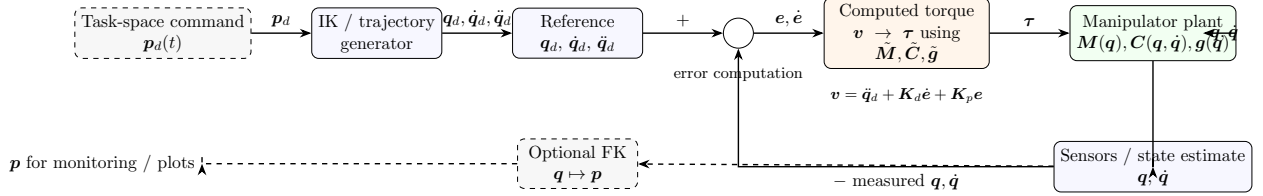


Figure 10: Computed-torque control signal flow. A task-space command can first be converted to a joint-space reference through IK, or $q_d(t)$ can be provided directly. The CT block uses the predicted model to generate torque τ , the manipulator returns measured joint state $q, \dot{q}$, and an optional FK block maps the measured state back to end-effector position for monitoring.

Motor torque limits (Phase 2):

Motor	Model	$T_{\max}$	Clip in code
Shoulder	AK80-kv60 (V1.x)	32 N·m	<code>np.clip(tau[0], -28, 28)</code>
Elbow	AK60-kv80 (V3.0)	15 N·m	<code>np.clip(tau[1], -12, 12)</code>

Listing 20: Computed Torque implementation (Phase 2 arm).

```

1 import numpy as np
2
3 # ---- Parameters (fill from identification) ----
4 L = np.array([0.335, 0.380]) # m
5 m = np.array([5.313, 0.764]) # kg (URDF; re-weigh on hardware)
6 g = 0.0 # horizontal XY plane: gravity acts in -Z, perpendicular to motion plane
7 T_MAX = np.array([28.0, 12.0]) # N.m [shoulder, elbow]
8
9 def inertia(q):
10     c2 = np.cos(q[1])
11     M11 = 4/3*m[0]*L[0]**2 + m[1]*(L[0]**2+4/3*L[1]**2+2*L[0]*L[1]*c2)
12     M12 = m[1]*(4/3*L[1]**2 + L[0]*L[1]*c2)
13     M22 = 4/3*m[1]*L[1]**2
14     return np.array([[M11, M12], [M12, M22]])
15
16 def coriolis(q, dq):
17     h = -m[1]*L[0]*L[1]*np.sin(q[1])
18     return np.array([[h*dq[1], h*(dq[0]+dq[1])], [-h*dq[0], 0.0]])
19
20 def gravity(q):
21     # Horizontal plane: gravity acts in -Z, perpendicular to XY plane of motion.
22     # No joint torque from gravity => g(q) = 0.
23     return np.zeros(2)
24
25 # ---- CT gains (omega_n = 12 rad/s, critically damped) ----
26 wn = 12.0
27 Kp = wn**2 * np.eye(2)
28 Kd = 2*wn * np.eye(2)
29

```



```

30 def ct_torque(q, dq, q_d, dqd, qdd_d):
31     e = q_d - q
32     de = dqd - dq
33     v = qdd_d + Kd @ de + Kp @ e
34     tau = inertia(q) @ v + coriolis(q, dq) @ dq + gravity(q)
35     return np.clip(tau, -T_MAX, T_MAX)

```

6.4 SEA cable compensation at the elbow

The elbow is driven via a cable spring (stiffness k_s) between the motor drum and the joint pulley. The spring extension is:

$$\delta = l_m - r_p q_2, \quad \dot{\delta} = \dot{l}_m - r_p \dot{q}_2, \quad (45)$$

where $l_m = r_p \theta_m$ is the cable displacement paid out by the drum. The cable force and resulting joint torque are:

$$F_{\text{cable}} = k_s \delta + b_c \dot{\delta}, \quad \tau_2^{\text{actual}} = r_p F_{\text{cable}}. \quad (46)$$

To invert this for a desired CT torque, compute the required cable displacement:

$$F_{\text{des}} = \frac{\tau_{2,\text{des}}}{r_p}, \quad l_m^{\text{des}} = r_p q_2^{\text{enc}} + \frac{F_{\text{des}} - b_c \dot{\delta}}{k_s}. \quad (47)$$

Listing 21: SEA cable compensation for the elbow (Phase 2).

```

1  # SEA parameters (from cable force-extension test on hardware)
2  K_S = 10.0 # N/m (measure on real cable)
3  B_C = 0.5 # N.s/m
4  R_P = 0.04775 # m (big-pulley radius)
5
6  def sea_compensation(tau_des_2, q2_enc, dq2_enc, l_m, dl_m):
7      """Convert desired elbow torque -> required cable displacement."""
8      delta = l_m - R_P * q2_enc
9      ddelta = dl_m - R_P * dq2_enc
10     F_des = tau_des_2 / R_P
11     l_m_des = R_P * q2_enc + (F_des - B_C * ddelta) / K_S
12     return l_m_des

```

△ Important

SEA bandwidth. Raise k_s (use stiffer cable / pre-tension) or lower ω_n if oscillatory torque tracking is observed on hardware.

6.5 Hybrid CT + inner MIT damping

Send the CT output as τ_{ff} and keep small on-board K_d to damp encoder noise without fighting the feedforward:

$$\tau_{\text{ff}} = \tau_{\text{CT}}, \quad K_p^{\text{MIT}} = 0, \quad K_d^{\text{MIT}} = 0.3\text{--}0.6. \quad (48)$$

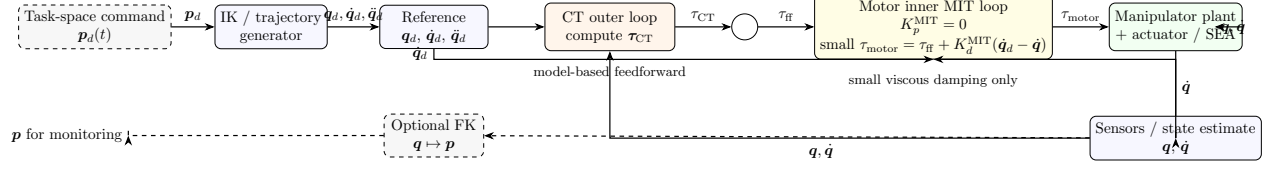


Figure 11: Hybrid CT signal flow. The outer loop computes the model-based torque τ_{CT} , which is sent as feedforward τ_{ff} to the motor's inner MIT loop. The MIT loop does not perform position control ($K_p^{MIT} = 0$); it only adds local velocity damping through the term $K_d^{MIT}(\dot{q}_d - \dot{q})$, producing the final motor torque τ_{motor} . The measured joint state $\mathbf{q}, \dot{\mathbf{q}}$ feeds back to the CT outer loop, while the measured velocity $\dot{\mathbf{q}}$ and desired velocity $\dot{\mathbf{q}}_d$ form the inner damping loop. IK and FK remain optional when the command originates in task space or when end-effector motion is monitored.

Listing 22: Hybrid CT: model feedforward + small on-board damping.

```

1 tau = ct_torque(q, dq, q_d, dqd, qdd_d)
2
3 # Shoulder: AK80-kv60 V1.x API
4 shoulder.write("mit_command", {"shoulder": q_d[0]},
5               kp=0.0, kd=0.4, dq_des=dqd[0], tau_ff=tau[0])
6 # Elbow: AK60-kv80 V3.0 API
7 elbow.write("mit_command", {"elbow": q_d[1]},
8            kp=0.0, kd=0.4, dq_des=dqd[1], tau_ff=tau[1])

```

6.6 Obtaining the real model from the robot in practice

For CT, we do not measure $\mathbf{M}(\mathbf{q})$, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$, and $\mathbf{g}(\mathbf{q})$ directly as matrices. Instead, we obtain the constant physical parameters of the arm and then build the model from them.

What we actually need. For the 2-DOF arm, the useful quantities are

$$L_1, L_2, m_1, m_2, l_{c1}, l_{c2}, I_1, I_2. \quad (49)$$

Once these are available, we can construct $\mathbf{M}(\mathbf{q})$, $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$, and $\mathbf{g}(\mathbf{q})$.

What is directly doable on the real robot?

- Measure L_1 and L_2 with a caliper or tape.
- Measure m_1 and m_2 by weighing each link assembly, including fasteners, brackets, adapters, and any mounted hardware that moves with the link.
- Measure pulley radius r_p directly.
- Measure cable stiffness k_s with a static force-extension test.
- Use CAD as the initial source for l_{c1} , l_{c2} , I_1 , and I_2 .

Recommended workflow.

1. Start from CAD values for CoM locations and moments of inertia.
2. Replace CAD masses and lengths with measured hardware values.
3. Load the resulting parameters into the robot model.
4. If tracking is not good enough, refine the uncertain parameters using logged motion and torque data.

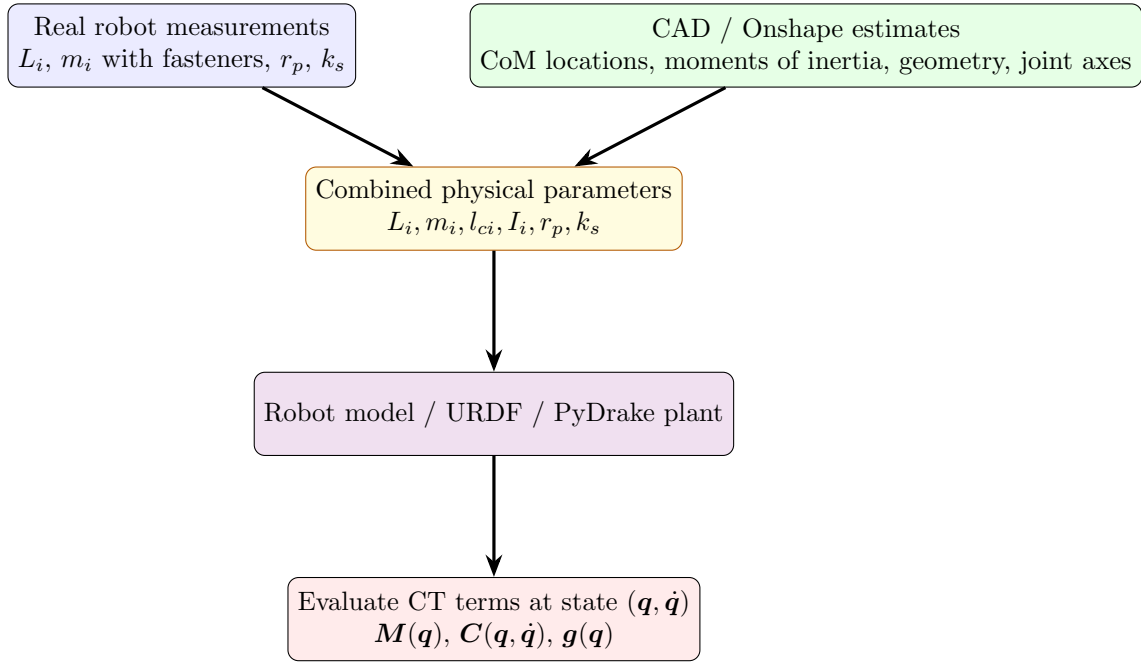


Figure 12: Practical flow for identifying the quantities needed to build $M(q)$, $C(q, \dot{q})$, and $g(q)$. Measured link masses should include the real moving hardware attached to each link, including fasteners and brackets.

What this means for M , C , and g . $M(q)$, $C(q, \dot{q})$, and $g(q)$ are state-dependent quantities, so they are not measured once and stored as constants. The constants are the physical parameters above. After those are known, the model can be evaluated at any joint state.

✓ Key Result

The practical goal is simple: measure what can be measured on the robot, take the remaining inertial quantities from CAD, and refine only if the controller needs better accuracy.

6.7 Using Onshape and PyDrake to obtain the dynamic model

If the arm already exists in Onshape, this is the cleanest route to a usable model for PyDrake.

Important distinction. The OBJ file is *not* the dynamic model by itself. The OBJ file mainly stores the mesh geometry used for visualisation and collision. The quantities needed for dynamics — masses, centres of mass, and inertia tensors — must appear in the URDF or SDF inertial tags. PyDrake reads those inertial properties from the robot description, not from the OBJ mesh alone.

What should come from CAD? From Onshape or CAD we should export:

- link geometry,
- link masses,
- centre-of-mass locations,
- inertia tensors,
- joint locations and joint axes.

These inertial values must be written into the URDF. The OBJ files are only used as mesh assets.

Practical pipeline.

Onshape/CAD $\implies$ URDF with inertial parameters $\implies$ load into PyDrake $\implies$ evaluate $\mathbf{M}(\mathbf{q})$ and bias term $\tau_{bias}(\mathbf{q}, \dot{\mathbf{q}})$ (50)

In this pipeline, OBJ conversion is only a supporting step so that Drake can load the mesh assets cleanly.

What does PyDrake calculate? Once the URDF is loaded into a `MultibodyPlant`, PyDrake can evaluate the robot dynamics at the current joint state. Conceptually, this means:

$$\mathbf{M}(\mathbf{q}) \leftarrow \text{mass matrix from the plant at } \mathbf{q}, \quad (51)$$

$$\tau_{bias}(\mathbf{q}, \dot{\mathbf{q}}) \leftarrow \text{bias term from the plant}, \quad (52)$$

where the bias term contains Coriolis and centrifugal effects, and also gravity if gravity is enabled. In our horizontal-plane case, gravity is typically set to zero or contributes no in-plane torque, so the bias term mainly represents the velocity-dependent effects associated with $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}}$.

What this means in practice. If the URDF inertial values are good, PyDrake can directly provide the dynamic terms needed for CT. If the CAD values are only approximate, then the Drake model is still useful, but its accuracy is limited by the parameters stored in the URDF.

Listing 23: Conceptual PyDrake route to obtain the dynamic terms from a URDF model.

```

1 plant_context = plant.CreateDefaultContext()
2 plant.SetPositions(plant_context, model_instance, q)
3 plant.SetVelocities(plant_context, model_instance, qd)
4
5 M = plant.CalcMassMatrixViaInverseDynamics(plant_context)
6 bias = plant.CalcBiasTerm(plant_context)
7
8 # Horizontal-plane arm: gravity contribution is zero or disabled.
9 # For CT, use M and bias directly in the feedforward torque calculation.
```

✓ Key Result

Use Onshape for geometry and inertial parameters, store them in the URDF, use OBJ only for meshes, and let PyDrake evaluate $\mathbf{M}(\mathbf{q})$ and the nonlinear bias term needed by the CT controller.

7 Phase 1 with Exo — Transparent Mode

★ Phase 1:

When to add the exo Once the manipulator tracks reliably under Phase 1 MIT PD (arm only, Section 5), mount the exosuit Damiao motors. In *transparent* mode they follow the elbow encoder angle so the cables stay slack and the arm experiences no additional loading. This step verifies correct exo motor direction and cable routing before activating co-contraction in Phase 2 with Exo.

7.1 Exo Damiao motors — transparent mode

The Damiao motor follows the elbow encoder angle with *zero* cable tension so it does not load the arm:

$$\tau_{exo} = 0, \quad q_{d,exo} = +q_2^{enc}, \quad K_p^{exo} = 0. \quad (53)$$

Listing 24: Exo transparent mode: Damiao motor follows AS5048A encoder.

```

1 # Read true elbow angle from absolute encoder
2 q2_enc = enc.read_angle_rad()
3
4 # Exo motor tracks joint angle -> zero cable extension -> zero torque
5 exo.write("mit_command",
6           {"exo": q2_enc},
7           kp=0.0, kd=0.05, tau_ff=0.0) # light viscous damp only

```

△ Important

Exo motor direction. Verify sign convention with slow manual elbow flexion before running the full loop: the cable should remain slack (no resistance felt by hand).

7.2 Complete 4-actuator Phase 1 loop (arm + transparent exo)

Listing 25: 4-actuator Phase 1 loop: MIT PD + transparent exo.

```

1 import time
2 import numpy as np
3 from cubemars.ak_v1.ak_v1_bus import CubeMarsAkV1Bench
4 from cubemars.ak_v3.ak_v3_bus import CubeMarsAkV3Bench
5 from damiao.damiao_bus import DamiaoBus
6 from as5048a import AS5048A
7 from motor_config import (
8     CubeMarsAkV1BusConfig, CubeMarsAkV1MotorConfig,
9     CubeMarsAkV3BusConfig, CubeMarsMotorConfig,
10    DamiaoBusConfig, DamiaoMotorConfig,
11 )
12 from encoder_config import AS5048AConfig
13
14 # ---- Hardware objects ----
15 shoulder = CubeMarsAkV1Bench(channel="can1", can_id=0x01,
16                             model="AK80-8", name="shoulder")
17 elbow = CubeMarsAkV3Bench(channel="can1", can_id=0x02,
18                           model="AK60-6", name="elbow")
19 exo = DamiaoBus(DamiaoBusConfig(
20     port="/dev/ttyACMO",
21     motors={"exo": DamiaoMotorConfig(can_id=0x01)},
22 ))
23 enc = AS5048A(AS5048AConfig(bus=0, device=0))
24
25 # ---- Phase-1 PD gains (no model) ----
26 KP_SHOULDER, KD_SHOULDER = 40.0, 1.2
27 KP_ELBOW, KD_ELBOW = 25.0, 0.8
28
29 def traj_phase1(t):
30     q_d = np.array([0.3*np.sin(0.3*t), 0.2*np.sin(0.5*t)])
31     dq_d = np.array([0.3*0.3*np.cos(0.3*t), 0.2*0.5*np.cos(0.5*t)])
32     return q_d, dq_d
33
34 DT = 0.01 # 100 Hz
35
36 with shoulder, elbow, exo, enc:
37     shoulder.enable()
38     elbow.enable()
39     exo.connect()
40     t0 = time.monotonic()
41     try:

```

```

42     while True:
43         t = time.monotonic() - t0
44         q2_enc = enc.read_angle_rad()
45         q1, dq1, _ = shoulder.read_state()["shoulder"]
46         _, dq2, _ = elbow.read_state()["elbow"]
47         q_d, dqd = traj_phase1(t)
48
49         shoulder.write("goal_position", {"shoulder": q_d[0]},
50                        kp=KP_SHOULDER, kd=KD_SHOULDER)
51         elbow.write("goal_position", {"elbow": q_d[1]},
52                    kp=KP_ELBOW, kd=KD_ELBOW)
53
54         # --- Exo transparent: follow joint angle, zero torque ---
55         exo.write("mit_command",
56                  {"exo": q2_enc},
57                  kp=0.0, kd=0.05, tau_ff=0.0)
58
59         time.sleep(max(0.0, DT - (time.monotonic() - t0 - t)))
60
61     except KeyboardInterrupt:
62         print("Stopping -- zeroing all torques")
63     finally:
64         shoulder.write("goal_torque", {"shoulder": 0.0})
65         elbow.write("goal_torque", {"elbow": 0.0})
66         exo.write("goal_torque", {"exo": 0.0})

```

7.3 Exo transparent commissioning checklist

1. **Exo transparent check:** manually flex the elbow through its full range of motion — no resistance should be felt from the exo cable.
2. **Direction check:** if resistance is felt in either direction, reverse the exo motor direction mapping and repeat.
3. **Cable slack at rest:** at zero velocity, verify cable tension reads near zero (compare exo encoder to AS5048A encoder).
4. **E-stop test:** Ctrl+C – all four motors coast to zero within one loop tick.

8 Phase 2 with Exo — CT + Co-contraction

★ Phase 2:

When to add the exo Once the arm tracks reliably under Phase 2 CT (arm only, Section 6), activate the exo Damiao motors. First confirm the transparent baseline from Section 7, then switch to co-contraction mode by commanding a symmetric angular offset $\Delta\theta$ on both exo cables. This adds programmable joint-level stiffness K_e at the elbow without changing the CT controller.

8.1 Exo co-contraction activation

The exo Damiao motors are commanded with a symmetric offset so that both cables develop equal pretension, adding co-contraction stiffness K_e at the elbow (`Exo_Impedance_Learning_PyDrake_Implementation`).

$$\frac{\theta_{mR}}{N} \rightarrow q_{2,\text{des}} + \Delta\theta, \quad \frac{\theta_{mL}}{N} \rightarrow -q_{2,\text{des}} + \Delta\theta, \quad (54)$$

where $\Delta\theta$ is the co-contraction offset [rad]. The effective joint stiffness is approximately $K_e \approx 2 k_s^{\text{exo}} r_p^2$.

Listing 26: Exo co-contraction activation (Phase 2 with exo).

```

1 DELTA_THETA = 0.10 # rad (co-contraction offset; tune empirically)
2 EXO_KP = 2.0; EXO_KD = 0.1
3
4 def exo_command(q2_des, activate, dth=DELTA_THETA):
5     if activate:
6         return q2_des + dth, -q2_des + dth
7     else:
8         return q2_des, -q2_des # transparent
9
10 q2_des_exo = q_d[1] # CT desired elbow angle
11 th_R, th_L = exo_command(q2_des_exo, activate=exo_on)
12 exo.write("mit_command",
13         {"exo_r": th_R, "exo_l": th_L},
14         kp=EXO_KP, kd=EXO_KD, tau_ff=0.0)

```

9 Trajectory Generation

9.1 Minimum-jerk point-to-point

$$\mathbf{p}_d(t) = \mathbf{p}_0 + (\mathbf{p}_f - \mathbf{p}_0) [10(t/T)^3 - 15(t/T)^4 + 6(t/T)^5]. \quad (55)$$

9.2 Circle trajectory

Matches CircleTrajectory in controller/trajectory_drake.py:

Listing 27: Circle trajectory + IK for hardware.

```

1 import numpy as np
2
3 def circle(t, center=np.array([0.40,0.20]), r=0.05, T=4.0):
4     th = 2*np.pi*t/T
5     p = center + r*np.array([np.cos(th), np.sin(th)])
6     pd = r*2*np.pi/T*np.array([-np.sin(th), np.cos(th)])
7     pdd = -r*(2*np.pi/T)**2*np.array([np.cos(th), np.sin(th)])
8     return p, pd, pdd
9
10 def ik(p, L):
11     D = np.clip((p@p-L[0]**2-L[1]**2)/(2*L[0]*L[1]), -1+1e-6, 1-1e-6)
12     q2 = np.arctan2(np.sqrt(1-D**2), D)
13     q1 = np.arctan2(p[1],p[0]) - np.arctan2(L[1]*np.sin(q2),
14                                             L[0]+L[1]*np.cos(q2))
15     return np.array([q1, q2])
16
17 def jacobian(q, L):
18     s12 = np.sin(q[0]+q[1]); c12 = np.cos(q[0]+q[1])
19     return np.array([[-L[0]*np.sin(q[0])-L[1]*s12, -L[1]*s12],
20                     [ L[0]*np.cos(q[0])+L[1]*c12, L[1]*c12]])

```


10 Controller Comparison

Controller	Model needed	Phase	Steady-state	Use case
MIT on-board PD	None	1	Gravity sag	Commissioning, gain tuning
PD + gravity comp.	$\mathbf{g}$ only	1.5	Eliminated	Slow Cartesian after m_i known
CT (joint)	Full $\mathbf{M}, \mathbf{C}, \mathbf{g}$	2	Eliminated	Trajectory tracking
CT + SEA compensation	Model + k_s	2	Eliminated	Compliant elbow cable
CT + exo co-contraction	Model + K_e	2+	Eliminated	Exo rehab stiffness
Cartesian PD (J-transp.)	$\mathbf{J} + \mathbf{g}$	2	Designed-in	Safe compliant interaction

11 Safety Constraints

- **Torque limits:** $|\tau_1| \leq 28 \text{ N}\cdot\text{m}$ (AK80-kv60), $|\tau_2| \leq 12 \text{ N}\cdot\text{m}$ (AK60-kv80), $|\tau_{\text{exo}}| \leq 8 \text{ N}\cdot\text{m}$ per Damiao. Always `np.clip` before sending.
- **Velocity limits:** $|\dot{q}_1| \leq 30 \text{ rad/s}$, $|\dot{q}_2| \leq 45 \text{ rad/s}$. Saturate $\dot{q}_d$ in trajectory generator.
- **Workspace limits:** $q_1 \in [-\pi/2, \pi]$, $q_2 \in [10^\circ, 170^\circ]$. Enforce in IK; reject unreachable Cartesian targets.
- **Encoder agreement check:** Each tick verify $|q_2^{\text{CAN}} - q_2^{\text{enc}}| < 0.15 \text{ rad}$. Larger discrepancy: switch to safe-hold (MIT PD, current position, low gains).
- **Thermal:** Monitor MOS temperature from `read_raw()`; pause if $> 70^\circ\text{C}$.
- **Emergency stop:** On SIGINT/Ctrl+C: zero torque on all four actuators via context-manager `--exit--` in each bus class.

12 Simulation Validation Bridge

All hardware controllers are first validated in the PyDrake simulation framework. The controller core (`rl/computed_torque_controller.py`) is **engine-agnostic** (pure NumPy, no Drake imports) so the same code runs in simulation and on hardware.

Hardware mode	Simulation analogue
Phase 1 MIT PD (no model)	<code>script...._with_exo_pydrake.py</code> (weak K_p/K_d)
Phase 2 CT (arm only)	<code>ComputedTorquePyDrakeImplementation.tex</code>
Phase 2 CT + SEA	<code>SEA_CablePyDrakeImplementation.tex, --sea-mode position</code>
Phase 2 CT + exo	<code>Exo_Impedance_Learning_PyDrake_Implementation.tex, --exo-activate</code>

Workflow.

1. Tune gains in PyDrake (Meshcat at <http://localhost:7000>).
2. Export to `controller_config.yaml`.
3. Load YAML on Jetson Orin for hardware deployment.
4. Compare logged trajectories with simulation via `compare_exo_pydrake_vs_isaac.py`.